TP n°4

Introduction au Procedural Language (PL/SQL) d'Oracle

Bouattay Ferdaous

Universite Sorbonne Paris Nord – USPN

Exercice 1

1.1 – Somme de deux entiers

```
SQL> @/Users/doussa/Desktop/BDD2eme/TP4/exo1.sql
Enter value for entier_a: 5
old   3:         a NUMBER := &entier_a;
new   3:         a NUMBER := 5;
Enter value for entier_b: 3
old   4:         b NUMBER := &entier_b;
new   4:         b NUMBER := 3;
La somme de 5 et 3 est : 8

PL/SQL procedure successfully completed.
```

```
doussa@Btys-MacBook TP4 % cat > /Users/doussa/Desktop/BDD2eme/TP4/exo1.sql << 'EOF'
SET SERVEROUTPUT ON;

DECLARE
    PROCEDURE calcul_somme IS
        a NUMBER := &entier_a;
        b NUMBER := &entier_b;
    BEGIN
        DBMS_OUTPUT.PUT_LINE('La somme de ' || a || ' et ' || b || ' est : ' || (a + b));
    END calcul_somme;

BEGIN
    calcul_somme;
END;
/
EOF
```

1.2 – Table de multiplication

```
doussa@Btys-MacBook TP4 % cat > /Users/doussa/Desktop/BDD2eme/TP4/exo1.sql << 'EOF'
SET SERVEROUTPUT ON;

DECLARE
    PROCEDURE table_multiplication IS
        nombre NUMBER := &saisir_nombre;
    BEGIN
        DBMS_OUTPUT.PUT_LINE('Table de multiplication de ' || nombre || ' :');
        FOR i IN 1..10 LOOP
            DBMS_OUTPUT.PUT_LINE(nombre || ' x ' || i || ' = ' || (nombre * i));
        END LOOP;
    END table_multiplication;

BEGIN
    table_multiplication;
END;
/
EOF
```

```
SQL> @/Users/doussa/Desktop/BDD2eme/TP4/exo1.sql
Enter value for saisir_nombre: 7
old   3:         nombre NUMBER := &saisir_nombre;
new   3:         nombre NUMBER := 7;
Table de multiplication de 7 :
7 x 1 = 7
7 x 2 = 14
7 x 3 = 21
7 x 4 = 28
7 x 5 = 35
7 x 6 = 42
7 x 7 = 49
7 x 8 = 56
7 x 9 = 63
7 x 10 = 70

PL/SQL procedure successfully completed.
```

1.3 – Fonction recursive puissance x^n

```
doussa@Btys-MacBook TP4 % cat > /Users/doussa/Desktop/BDD2eme/TP4/exo1.sql << 'EOF'
SET SERVEROUTPUT ON;

DECLARE
    v_base     NUMBER;
    v_exposant NUMBER;
    v_resultat NUMBER;

    FUNCTION puissance(x NUMBER, n NUMBER) RETURN NUMBER IS
    BEGIN
        IF n = 0 THEN
            RETURN 1;
        ELSE
            RETURN x * puissance(x, n-1);
        END IF;
    END puissance;

BEGIN
    v_base     := &entrez_base;
    v_exposant := &entrez_exposant;
    v_resultat := puissance(v_base, v_exposant);
    DBMS_OUTPUT.PUT_LINE(v_base || '^' || v_exposant || ' = ' || v_resultat);
END;
/
EOF
```

```
SQL> @/Users/doussa/Desktop/BDD2eme/TP4/exo1.sql
Enter value for entrez_base: 5
old  16:     v_base     := &entrez_base;
new  16:     v_base     := 5;
Enter value for entrez_exposant: 3
old  17:     v_exposant := &entrez_exposant;
new  17:     v_exposant := 3;
5^3 = 125

PL/SQL procedure successfully completed.
```

1.4 – Factorielle stockee dans `resultatFactoriel`

```
SQL> @/Users/doussa/Desktop/BDD2eme/TP4/exo1.sql
Enter value for saisir_nombre: 5
old   2:     v_nombre       NUMBER := &saisir_nombre;
new   2:     v_nombre       NUMBER := 5;
        INSERT INTO resultatFactoriel(nombre, factorielle)
                    *
ERROR at line 33:
ORA-06550: line 33, column 21:
PL/SQL: ORA-00942: table or view does not exist
ORA-06550: line 33, column 9:
PL/SQL: SQL Statement ignored
ORA-06550: line 39, column 20:
PL/SQL: ORA-00942: table or view does not exist
ORA-06550: line 39, column 13:
PL/SQL: SQL Statement ignored
Help: https://docs.oracle.com/error-help/db/ora-06550/
```

```
SQL> @/Users/doussa/Desktop/BDD2eme/TP4/exo1.sql
Table resultatFactoriel creee avec succes.

PL/SQL procedure successfully completed.

Enter value for saisir_nombre: 5
old   2:     v_nombre NUMBER := &saisir_nombre;
new   2:     v_nombre NUMBER := 5;
5! = 120
Resultat stocke avec succes.

PL/SQL procedure successfully completed.
```

```
SQL> SELECT * FROM resultatFactoriel;

    NOMBRE FACTORIELLE
---------- -----------
DATE_CALCUL
---------------------------------------------------------------------------
         5         120
13-APR-26 12.35.04.000202 PM
```

```
SQL> @/Users/doussa/Desktop/BDD2eme/TP4/exo1.sql
Table resultatsFactoriels creee avec succes.

PL/SQL procedure successfully completed.

Calcul des factorielles de 1 a 20:
1! = 1
2! = 2
3! = 6
4! = 24
5! = 120
6! = 720
7! = 5040
8! = 40320
9! = 362880
10! = 3628800
11! = 39916800
12! = 479001600
13! = 6227020800
14! = 87178291200
15! = 1307674368000
16! = 20922789888000
17! = 355687428096000
18! = 6402373705728000
19! = 121645100408832000
20! = 2432902008176640000
Toutes les factorielles ont ete calculees et stockees.

PL/SQL procedure successfully completed.
```

1.5 – Factorielles des 20 premiers entiers dans `resultatsFactoriels`

```
doussa@Btys-MacBook TP4 % cat > /Users/doussa/Desktop/BDD2eme/TP4/exo2.sql << 'EOF'
SET SERVEROUTPUT ON;

CREATE TABLE emp (
    matr    NUMBER(10)   NOT NULL,
    nom     VARCHAR2(50) NOT NULL,
    sal     NUMBER(7,2),
    adresse VARCHAR2(96),
    dep     NUMBER(10)   NOT NULL,
    CONSTRAINT emp_pk PRIMARY KEY (matr)
);

COMMIT;
EOF
```

```
SQL> @/Users/doussa/Desktop/BDD2eme/TP4/exo2.sql

Table created.
```

Exercice 2

Creation de la table `emp` :

```
doussa@Btys-MacBook TP4 % cat > /Users/doussa/Desktop/BDD2eme/TP4/exo2.sql << 'EOF'
SET SERVEROUTPUT ON;

DECLARE
    v_employe emp%ROWTYPE;
BEGIN
    v_employe.matr    := 4;
    v_employe.nom     := 'Youcef';
    v_employe.sal     := 2500;
    v_employe.adresse := 'avenue Anatole France';
    v_employe.dep     := 92000;

    INSERT INTO emp VALUES v_employe;
    COMMIT;
    DBMS_OUTPUT.PUT_LINE('Employe ajoute avec succes : ' || v_employe.nom);
EXCEPTION
    WHEN DUP_VAL_ON_INDEX THEN
        DBMS_OUTPUT.PUT_LINE('Erreur : Matricule ' || v_employe.matr || ' existe deja');
        ROLLBACK;
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Erreur : ' || SQLERRM);
        ROLLBACK;
END;
/
EOF
```

```
SQL> @/Users/doussa/Desktop/BDD2eme/TP4/exo2.sql
Employe ajoute avec succes : Youcef

PL/SQL procedure successfully completed.
```

2.1 – Insertion d'un employe avec `%ROWTYPE`

```
SQL> SELECT * FROM emp;

      MATR NOM                                                 SAL
---------- ------------------------------------------------ ----------
ADRESSE
--------------------------------------------------------------------------------
       DEP
----------
         4 Youcef                                             2500
avenue Anatole France
     92000
```

```
SQL> @/Users/doussa/Desktop/BDD2eme/TP4/exo2.sql
Enter value for saisir_departement: 10
old   2:     v_dep      NUMBER := &saisir_departement;
new   2:     v_dep      NUMBER := 10;
Nombre d employes supprimes du departement 10 : 0

PL/SQL procedure successfully completed.
```

```
SQL> @/Users/doussa/Desktop/BDD2eme/TP4/exo2.sql
total : 2500

PL/SQL procedure successfully completed.
```

2.2 – Suppression par departement avec SQL%ROWCOUNT

```
SQL> @/Users/doussa/Desktop/BDD2eme/TP4/exo2.sql
Salaire moyen : 2500

PL/SQL procedure successfully completed.
```

2.3 – Somme des salaires avec curseur explicite et boucle LOOP

```
doussa@Btys-MacBook TP4 % cat > /Users/doussa/Desktop/BDD2eme/TP4/exo2.sql << 'EOF'
SET SERVEROUTPUT ON;

DECLARE
    v_salaire EMP.sal%TYPE;
    v_total   EMP.sal%TYPE := 0;
    v_count   NUMBER := 0;
    v_moyenne NUMBER;
    CURSOR c_salaires IS
        SELECT sal FROM emp;
BEGIN
    OPEN c_salaires;
    LOOP
        FETCH c_salaires INTO v_salaire;
        EXIT WHEN c_salaires%NOTFOUND;
        IF v_salaire IS NOT NULL THEN
            v_total := v_total + v_salaire;
            v_count := v_count + 1;
        END IF;
    END LOOP;
    CLOSE c_salaires;

    IF v_count > 0 THEN
        v_moyenne := v_total / v_count;
        DBMS_OUTPUT.PUT_LINE('Salaire moyen : ' || ROUND(v_moyenne, 2));
    ELSE
        DBMS_OUTPUT.PUT_LINE('Aucun salaire valide a calculer');
    END IF;
EXCEPTION
    WHEN OTHERS THEN
        IF c_salaires%ISOPEN THEN
            CLOSE c_salaires;
        END IF;
        DBMS_OUTPUT.PUT_LINE('Erreur : ' || SQLERRM);
END;
/
EOF
```

```
SQL> @/Users/doussa/Desktop/BDD2eme/TP4/exo2.sql
Somme totale des salaires : 2500

PL/SQL procedure successfully completed.

Salaire moyen : 2500

PL/SQL procedure successfully completed.
```

2.4 – Salaire moyen avec curseur explicite et boucle LOOP

```
SQL> @/Users/doussa/Desktop/BDD2eme/TP4/exo2.sql
Employes du departement 92000:
- Youcef
Employes du departement 75000:

PL/SQL procedure successfully completed.
```

2.5 – Somme et moyenne avec boucle FOR IN

```
SQL> @/Users/doussa/Desktop/BDD2eme/TP4/exo3.sql

Table created.


Commit complete.


Package created.


Package body created.
```

2.6 – Curseur parametree par departement

```
SQL> SELECT * FROM clients;

        ID NOM
---------- --------------------------------------------------
EMAIL
--------------------------------------------------------------------------------
         1 Alice
alice@email.com

         2 Bob
bob@email.com
```

Exercice 3 – Package `gestion_clients`

Creation du package avec deux procedures surchargees `ajouter_client` : la premiere prend les attributs individuellement, la seconde prend un `%ROWTYPE`. Les exceptions sont levees avec `RAISE_APPLICATION_ERROR`.

```
SQL> SELECT * FROM clients;

        ID NOM
---------- --------------------------------------------------
EMAIL
--------------------------------------------------------------------------------
         1 Alice
alice@email.com

         2 Bob
bob@email.com

         3 Alice
alice@email.com


        ID NOM
---------- --------------------------------------------------
EMAIL
--------------------------------------------------------------------------------
         4 Bob
bob@email.com
```

Test de la version 1 :

```
SQL> EXEC gestion_clients.ajouter_client(3, 'Alice', 'alice@email.com');
Client ajoute (version 1)

PL/SQL procedure successfully completed.
```

Test de la version 2 :

```
SQL> DECLARE
    v_client clients%ROWTYPE;
BEGIN
    v_client.id    := 4;
    v_client.nom   := 'Bob';
    v_client.email := 'bob@email.com';
    gestion_clients.ajouter_client(v_client);
END;
```

```
Client ajoute (version 2)

PL/SQL procedure successfully completed.
```